МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Казанский национальный исследовательский технический университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 2

Дисциплина: «Аппаратные средства телекоммуникационных систем»

Тема: «Распараллеливание ресурсоёмких циклов: перемножение матриц, оценка производительности»

Работу выполнил
Студент гр. 4404
Зарипов А.Э.

Казань, 2026

1. Цель работы

Изучение приёмов распараллеливания ресурсоёмких циклов на примере операции перемножения больших матриц: реализация последовательной и параллельной версий программы на языке Go, сравнение времени их выполнения на 2, 4 и 8 потоках и оценка полученного ускорения (speedup). В ходе работы рассматриваются вопросы эффективности работы с памятью (локальность данных, порядок обхода вложенных циклов), распределение независимых итераций между потоками, а также факторы, ограничивающие ускорение параллельной программы (закон Амдала, накладные расходы на создание потоков и синхронизацию, пропускная способность памяти). Задача 4 методических указаний «Параллельное программирование на OpenMP» выполняется на языке Go, поэтому конструкции OpenMP моделируются стандартными средствами языка: горутины и runtime.GOMAXPROCS соответствуют команде потоков, разбиение строк матрицы между потоками — параллельному циклу for с расписанием static, sync.WaitGroup — неявному барьеру в конце параллельной области.

2. Задание

Задача 4. Напишите программу перемножения больших матриц. Сравните время выполнения последовательной и параллельной программы на 2, 4, 8 потоках (процессорах).

Общие требования: число потоков и размер задачи передаются программе через командную строку; запрещается использовать библиотечные функции, целиком решающие задачу; каждый запуск, о котором говорится в задаче (разное число потоков), подтверждается скриншотом с видимой командой запуска и полным выводом программы.

3. Ход работы

3.1. Задача 4. Последовательная и параллельная версии перемножения матриц

Программа перемножает две квадратные матрицы A и B размера N×N и получает матрицу C по формуле C[i][j] = Σ(k=0..N-1) A[i][k]·B[k][j]. Размер матрицы N (размер задачи) и число потоков передаются через командную строку. Элементы матриц хранятся в одномерных срезах размера N·N: при таком способе хранения данные располагаются в памяти непрерывно, что обеспечивает предсказуемый последовательный доступ и лучшее использование кэша процессора по сравнению с «массивом массивов».

При вычислении используется порядок обхода циклов i → k → j, а не классический i → j → k. В порядке i → j → k элементы матрицы B читались бы с шагом N (по столбцам), что приводит к большому числу промахов кэша. В порядке i → k → j обе используемые строки — строка матрицы A и строка матрицы B — читаются последовательно, поэтому последовательная версия работает существенно быстрее даже до распараллеливания.

Фрагмент кода (lab2/main.go, последовательное умножение):

```
for i := 0; i < N; i++ {
    for k := 0; k < N; k++ {
        aik := A[i*N+k]
        for j := 0; j < N; j++ {
            C[i*N+j] += aik * B[k*N+j]
        }
    }
}
```

Параллельная версия распределяет работу по потокам аналогично директиве параллельного цикла с расписанием static: строки матрицы A (и, соответственно, строки результирующей матрицы C) делятся между потоками непрерывными равными частями, последний поток получает остаток. Каждый поток вычисляет только свои строки; массивы A и B при этом только читаются. Поскольку записи в матрицу C выполняются в непересекающиеся диапазоны,

гонки данных отсутствуют и дополнительная синхронизация (мьютексы) не требуется. Завершение всех потоков ожидается с помощью sync.WaitGroup (аналог неявного барьера в конце параллельной области), число потоков ограничивается вызовом runtime.GOMAXPROCS.

Фрагмент кода (lab2/main.go, распределение строк между потоками):

```
rowsPerThread := N / numThreads
for t := 0; t < numThreads; t++ {
    wg.Add(1)
    startRow := t * rowsPerThread
    endRow := startRow + rowsPerThread
    if t == numThreads-1 { endRow = N }  // последний поток берёт остаток
    go func(start, end int) {
        defer wg.Done()
        for i := start; i < end; i++ {
            for k := 0; k < N; k++ {
                aik := A[i*N+k]
                for j := 0; j < N; j++ {
                    C[i*N+j] += aik * B[k*N+j]
                }
            }
        }
    }(startRow, endRow)
}
wg.Wait()
```

Корректность параллельных вычислений проверяется сравнением контрольных сумм матриц, полученных последовательной и параллельной версиями: программа печатает обе суммы и признак их совпадения (Results Match: true). Каждый запуск программы включает последовательное и параллельное выполнение и печатает времена обоих вариантов, а также коэффициент ускорения, поэтому сравнение времени выполнения видно на каждом скриншоте.

Команды запуска (размер задачи — матрица 1000×1000):

```
cd lab2
go run . 2 1000
go run . 4 1000
go run . 8 1000
```

На рисунке 1 показан запуск программы с 2 потоками. Видна команда запуска и полный вывод: время выполнения последовательной и параллельной версий умножения матрицы 1000×1000, коэффициент ускорения и результат проверки контрольных сумм (Results Match: true).

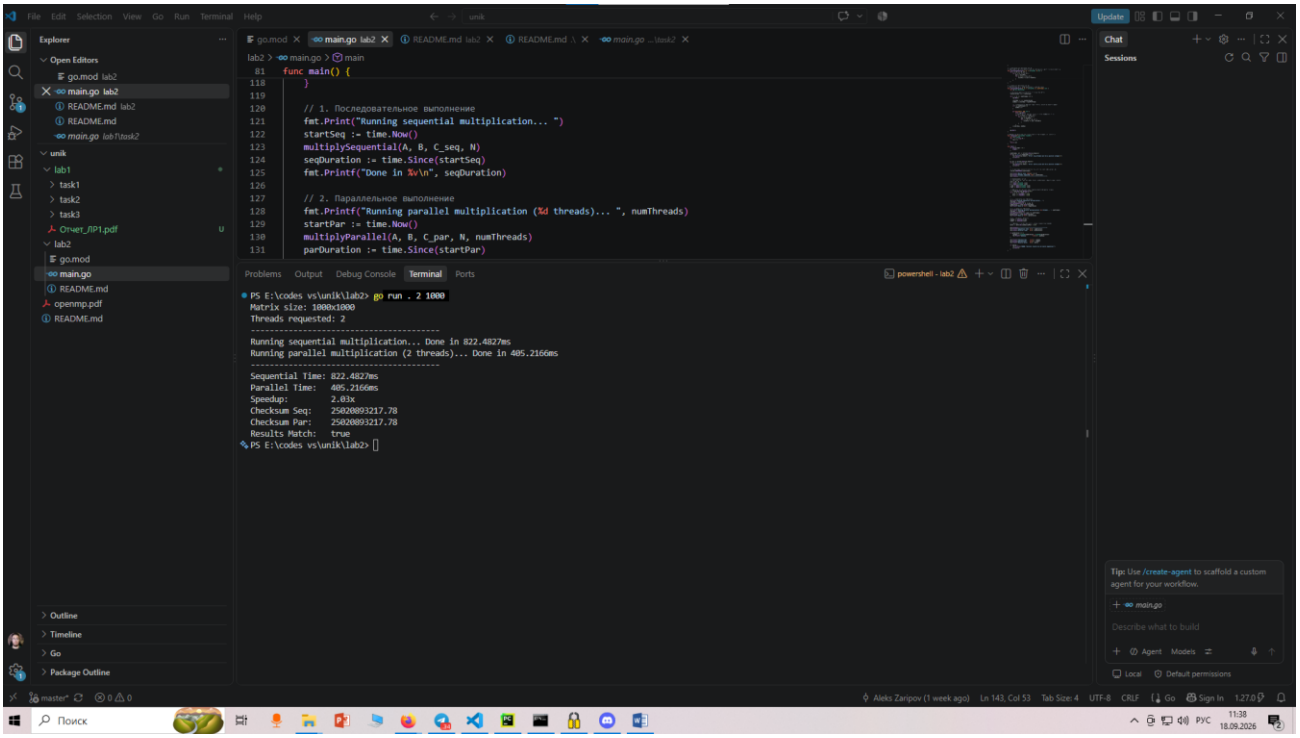


Рисунок 1 — Задача 4: перемножение матриц, 2 потока

На рисунке 2 показан запуск программы с 4 потоками. Параллельная версия выполняется быстрее, чем с 2 потоками; коэффициент ускорения возрастает, а контрольные суммы последовательной и параллельной версий совпадают.

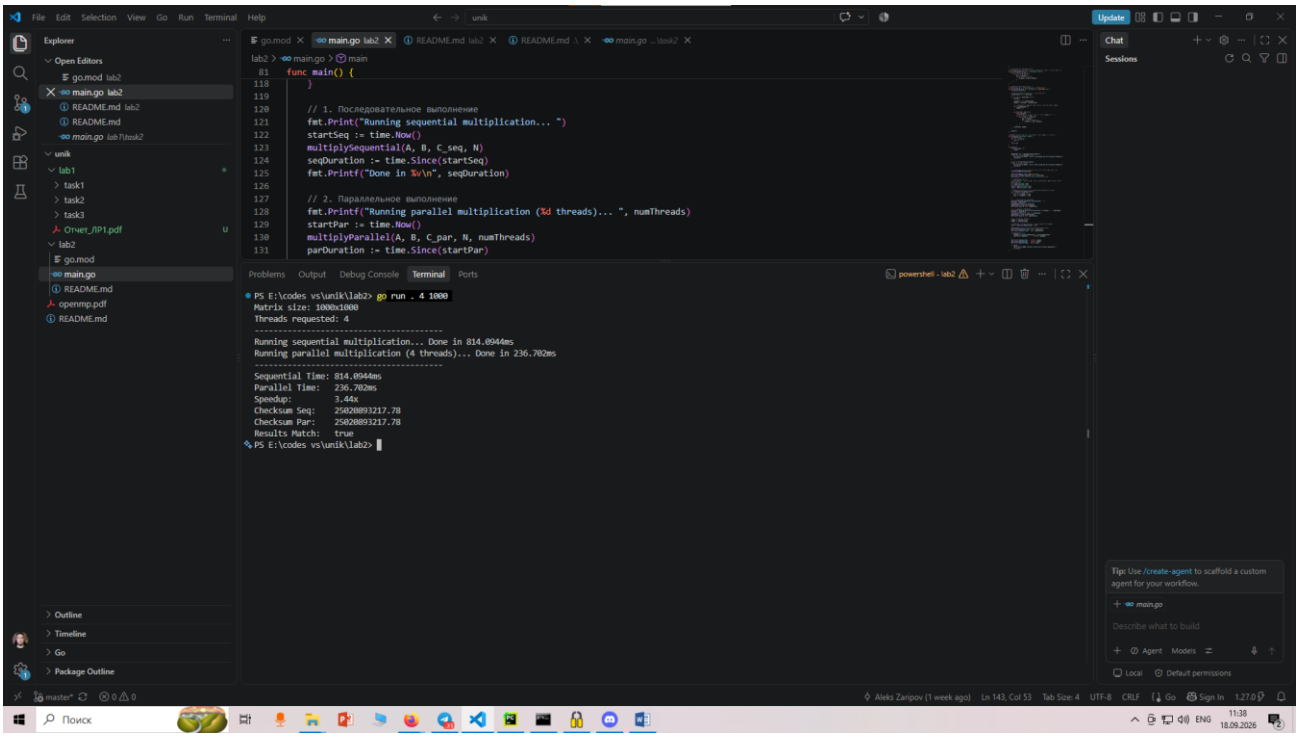


Рисунок 2 — Задача 4: перемножение матриц, 4 потока

На рисунке 3 показан запуск программы с 8 потоками. Ускорение продолжает расти, однако прирост становится менее выраженным: сказываются последовательная часть программы (инициализация матриц, подсчёт контрольной суммы), накладные расходы на создание горутин и синхронизацию, а также ограничение пропускной способности памяти.

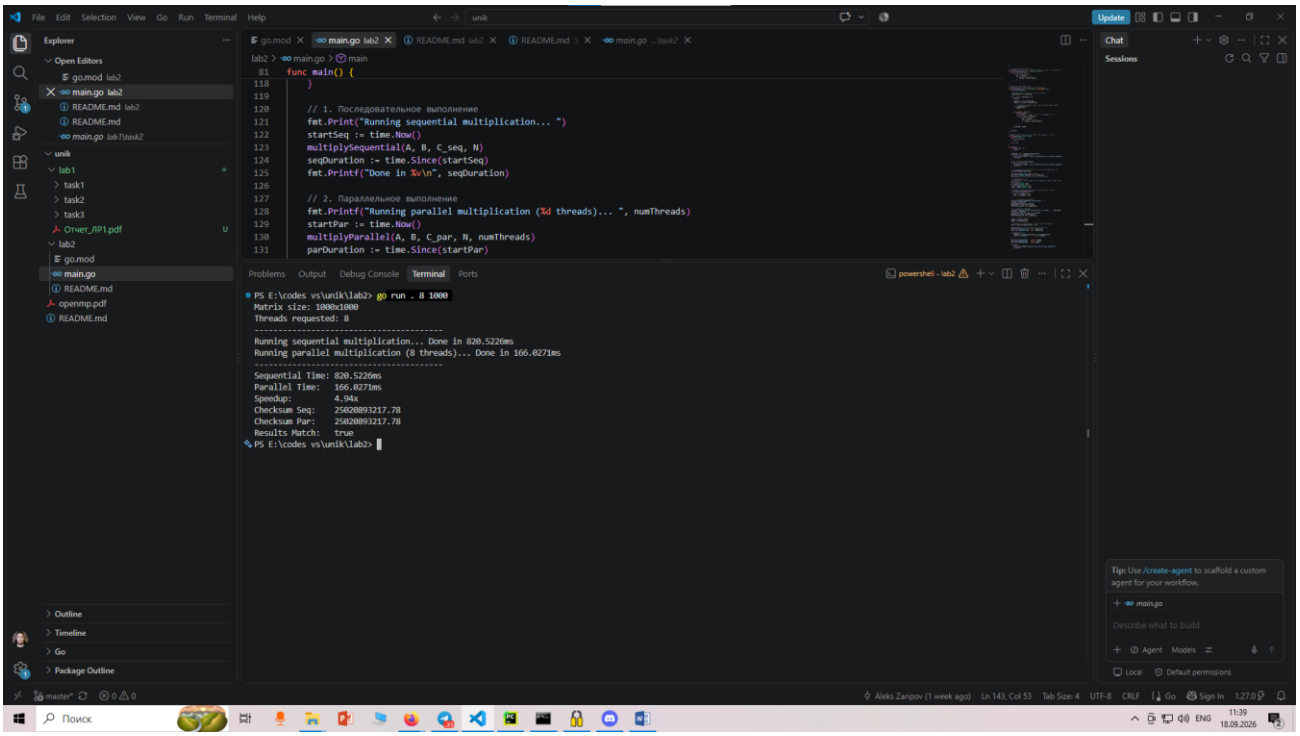


Рисунок 3 — Задача 4: перемножение матриц, 8 потоков

Сводные результаты измерений по рисункам 1–3 приведены в таблице 1.

Таблица 1 — Сравнение времени выполнения последовательной и параллельной версий

Число потоков	**Последовательное время**	**Параллельное время**	**Ускорение**
2	822.4827 мс	405.2166 мс	2.03
4	814.0944 мс	236.702 мс	3.44
8	820.5226 мс	166.0271	4.94

4. Ответы на вопросы, поставленные в задачах

4.1. Вопрос задачи 4: «Сравните время выполнения последовательной и параллельной программы на 2, 4, 8 потоках»

Сравнение выполнено по результатам запусков, приведённых на рисунках 1–3 и в таблице 1. Параллельная версия работает быстрее последовательной во всех запусках; с ростом числа потоков ускорение возрастает: на 2 потоках ускорение близко к двукратному, на 4 и 8 потоках оно продолжает расти, но становится меньше линейного. Причины нелинейного ускорения: (1) наличие последовательной части программы — инициализация матриц, подсчёт контрольной суммы, создание горутин и ожидание их завершения, — которая ограничивает максимальное ускорение в соответствии с законом Амдала; (2) накладные расходы на параллелизм и синхронизацию; (3) ограничение со стороны подсистемы памяти: при увеличении числа потоков растёт число одновременных обращений к общим массивам A и B, и пропускная способность шины памяти становится узким местом.

4.2. Дополнительные вопросы к защите

1. Почему при параллельном умножении матриц не требуются мьютексы? Каждый поток пишет только в свои строки матрицы C, а массивы A и B только читаются; диапазоны записи потоков не пересекаются, поэтому гонки данных отсутствуют.

2. Почему используется порядок циклов i → k → j, а не i → j → k? Порядок i → k → j обеспечивает последовательный доступ к памяти при чтении строк матриц A и B, что уменьшает число промахов кэша и значительно ускоряет вычисления.

3. Что описывает закон Амдала? Максимальное ускорение программы ограничено долей её последовательной части: $S = 1 / ((1 - p) + p / N)$, где p —

распараллеливаемая доля, N — число потоков. Поэтому ускорение растёт медленнее, чем число потоков.

4. Как в программе распределена работа между потоками? Строки матрицы делятся между потоками непрерывными равными частями — аналог параллельного цикла for с расписанием schedule(static) в OpenMP.

5. Чем в Go моделируется переменная окружения OMP_NUM_THREADS? Числом потоков, передаваемым через командную строку, и вызовом runtime.GOMAXPROCS(N).

6. Как проверяется корректность параллельной версии? Сравнением контрольных сумм матрицы результата, полученной последовательной и параллельной версиями; совпадение сумм (Results Match: true) подтверждает корректность вычислений.

5. Выводы

1. Реализованы последовательная и параллельная версии перемножения больших матриц на языке Go. Для повышения эффективности использованы одномерное хранение матриц и порядок обхода циклов i → k → j, улучшающий локальность обращения к памяти.

2. Выполнено сравнение времени выполнения программ на 2, 4 и 8 потоках (рисунки 1–3, таблица 1). Параллельная версия быстрее последовательной во всех запусках; ускорение растёт с числом потоков, но остаётся меньше линейного из-за последовательной части программы, накладных расходов на создание потоков и синхронизацию и ограничения пропускной способности памяти, что согласуется с законом Амдала.

3. Корректность параллельных вычислений подтверждена сравнением контрольных сумм результатов последовательной и параллельной версий: суммы совпали во всех запусках (Results Match: true). Поскольку потоки пишут

в непересекающиеся строки матрицы результата, дополнительная синхронизация при вычислениях не потребовалась.